Python for Data Analytics

Week 1 Assignment: Python Fundamentals

Submitted by: Deepanshu
University: University of Delhi (B.A. Economics)

Contents

Task 1: Python Basics (10 Marks)
Task 2: Variables & Data Types (10 Marks)
Task 3: Operators (10 Marks)
Task 4: Conditional Statements (10 Marks)
Task 5: Loops (10 Marks)
Task 6: Functions (10 Marks)
Task 7: Strings & Collections (15 Marks)
Task 8: Basic File Handling (10 Marks)
Task 9: Mini Project - Student Record Management System (15 Marks)

Google Drive link (all .py files): *[paste your shared Drive link here]*

Task 1: Python Basics (10 Marks)

Explanation

Prints a welcome banner, then uses input() to read the Name, College and Branch. The values are shown back using f-strings for neat, aligned formatted output.

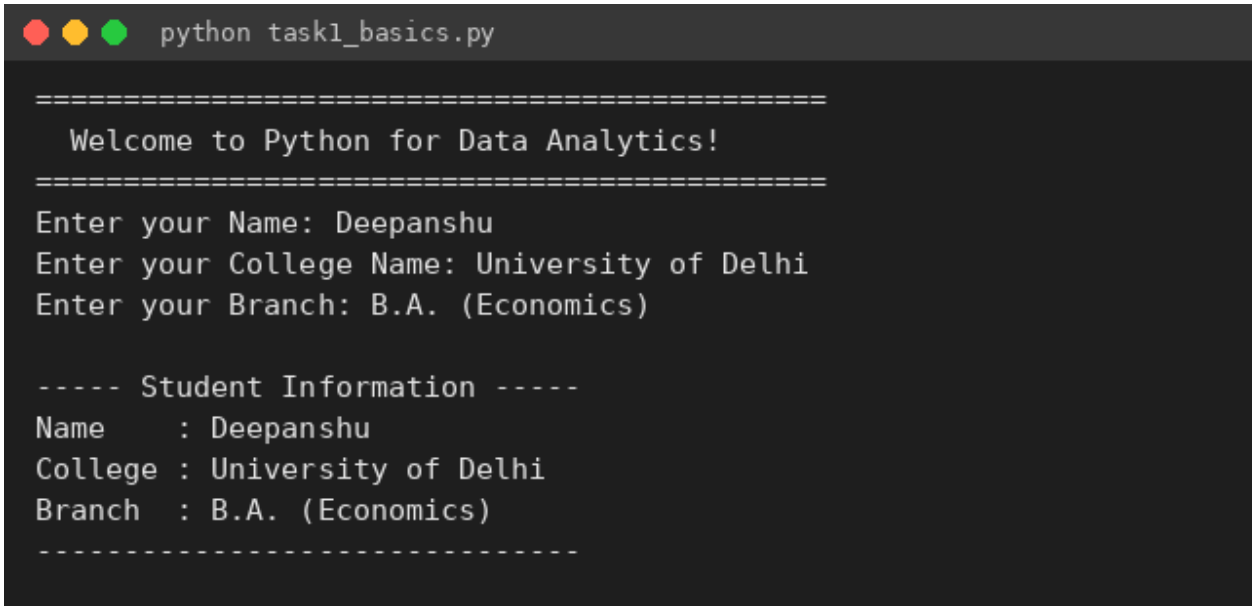
Source Code (task1_basics.py)

```
# Task 1: Python Basics - Welcome message and formatted student info
print("=" * 45)
print("  Welcome to Python for Data Analytics!")
print("=" * 45)

name = input("Enter your Name: ")
college = input("Enter your College Name: ")
branch = input("Enter your Branch: ")

print("\n----- Student Information -----")
print(f"Name    : {name}")
print(f"College : {college}")
print(f"Branch  : {branch}")
print("-------------------------------")
```

Output Screenshot



Task 2: Variables & Data Types (10 Marks)

Explanation

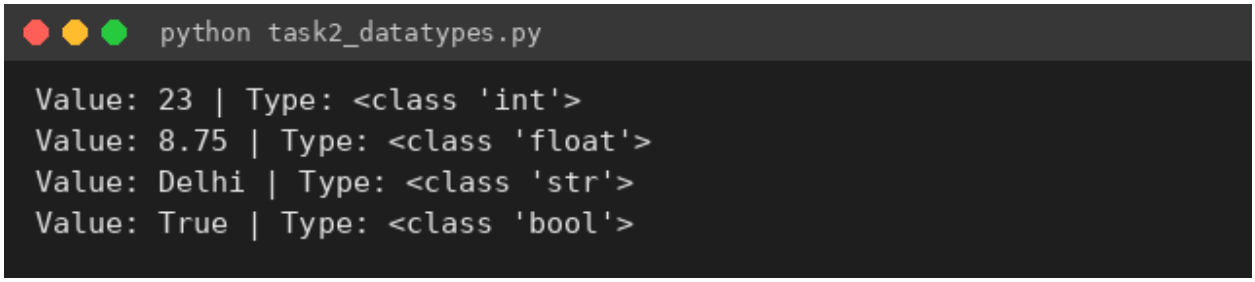
Declares an integer, a float, a string and a boolean. Each value is printed along with its data type using the built-in type() function.

Source Code (task2_datatypes.py)

```
# Task 2: Variables & Data Types
age = 23                # Integer
cgpa = 8.75             # Float
city = "Delhi"          # String
is_student = True       # Boolean

print("Value:", age, "| Type:", type(age))
print("Value:", cgpa, "| Type:", type(cgpa))
print("Value:", city, "| Type:", type(city))
print("Value:", is_student, "| Type:", type(is_student))
```

Output Screenshot



Task 3: Operators (10 Marks)

Explanation

Reads two numbers and applies the arithmetic operators +, -, *, / and %. A zero check prevents a ZeroDivisionError for division and modulus.

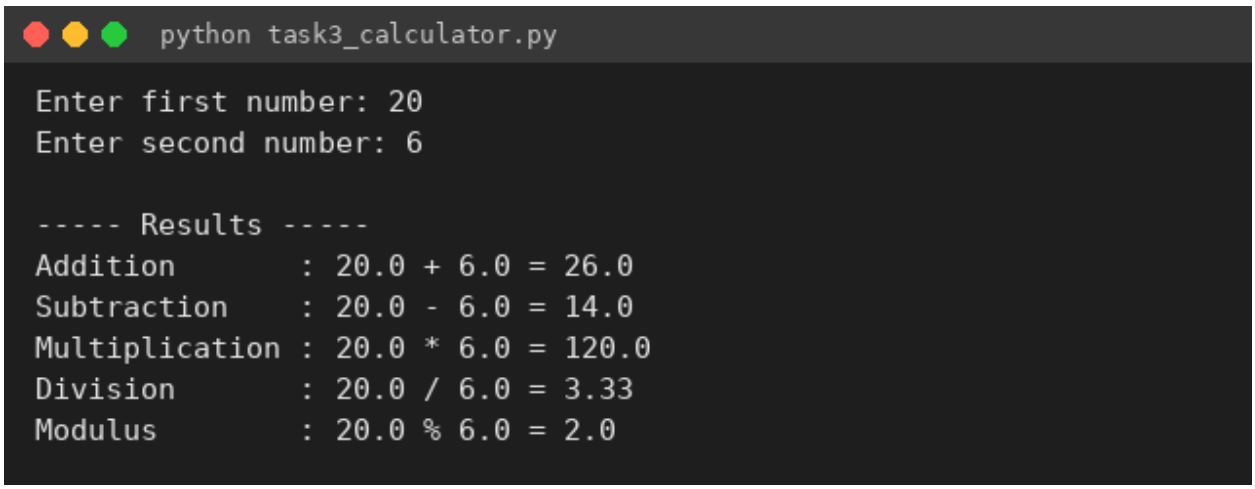
Source Code (task3_calculator.py)

```
# Task 3: Operators - Simple Calculator
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print("\n----- Results -----")
print(f"Addition       : {num1} + {num2} = {num1 + num2}")
print(f"Subtraction    : {num1} - {num2} = {num1 - num2}")
print(f"Multiplication : {num1} * {num2} = {num1 * num2}")

if num2 != 0:
    print(f"Division       : {num1} / {num2} = {num1 / num2:.2f}")
    print(f"Modulus        : {num1} % {num2} = {num1 % num2}")
else:
    print("Division       : Not possible (divide by zero)")
    print("Modulus        : Not possible (divide by zero)")
```

Output Screenshot



Task 4: Conditional Statements (10 Marks)

Explanation

Takes marks as input and uses an if / elif / else ladder to print the grade: 90+ is A, 75-89 is B, 60-74 is C, below 60 is Fail. Marks outside 0-100 are rejected.

Source Code (task4_grades.py)

```
# Task 4: Conditional Statements - Grade calculator
marks = float(input("Enter your marks (0-100): "))

if marks < 0 or marks > 100:
    print("Invalid marks! Enter a value between 0 and 100.")
elif marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
elif marks >= 60:
    print("Grade: C")
else:
    print("Grade: Fail")
```

Output Screenshot



Task 5: Loops (10 Marks)

Explanation

Uses a for loop to print 1 to 20, a for loop to print the multiplication table of a user-given number, and a while loop with a counter to print even numbers from 1 to 50.

Source Code (task5_loops.py)

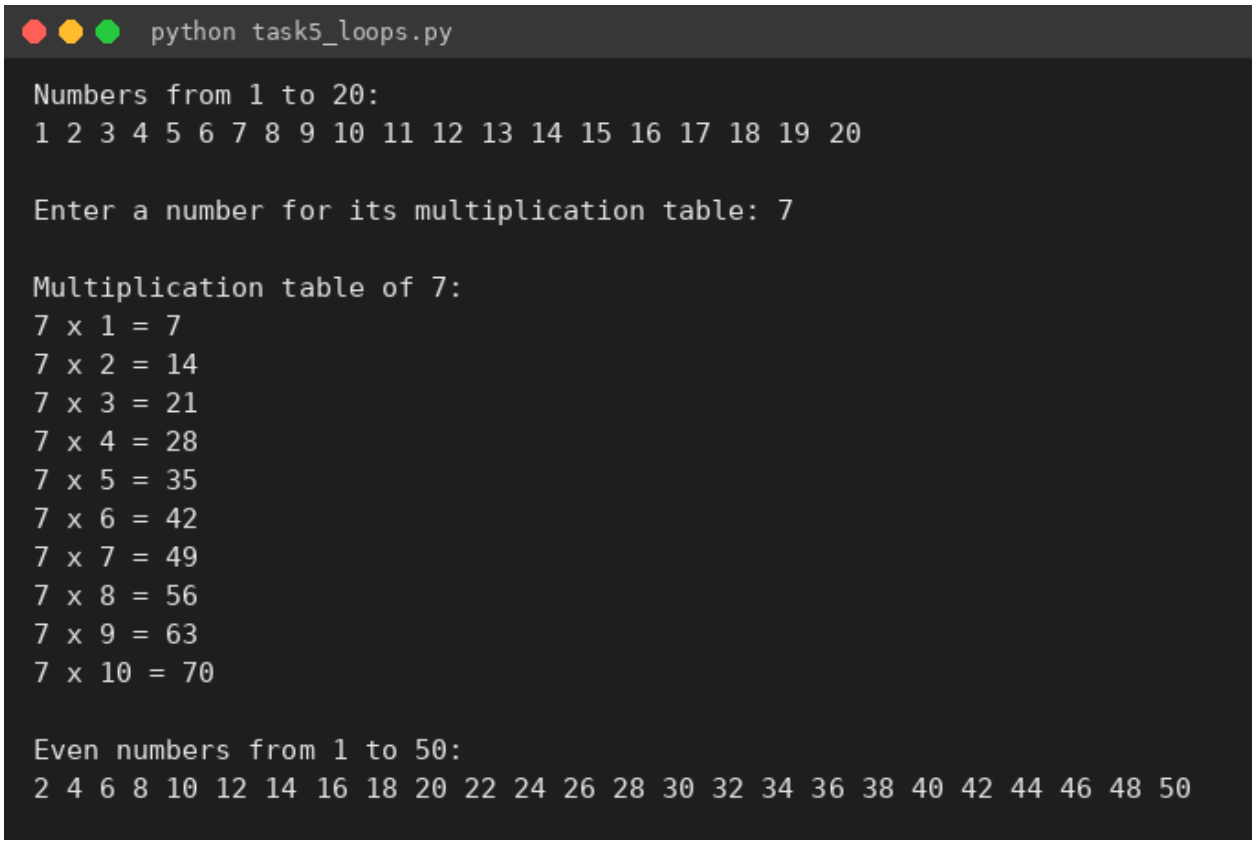
```
# Task 5: Loops

# 1. Numbers 1 to 20 using for loop
print("Numbers from 1 to 20:")
for i in range(1, 21):
    print(i, end=" ")
print("\n")

# 2. Multiplication table of any number
n = int(input("Enter a number for its multiplication table: "))
print(f"\nMultiplication table of {n}:")
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")

# 3. Even numbers 1 to 50 using while loop
print("\nEven numbers from 1 to 50:")
j = 1
while j <= 50:
    if j % 2 == 0:
        print(j, end=" ")
    j += 1
print()
```

Output Screenshot



Task 6: Functions (10 Marks)

Explanation

Defines square(num) and average(a, b, c) with return values. Both are called with numbers entered by the user.

Source Code (task6_functions.py)

```
# Task 6: Functions

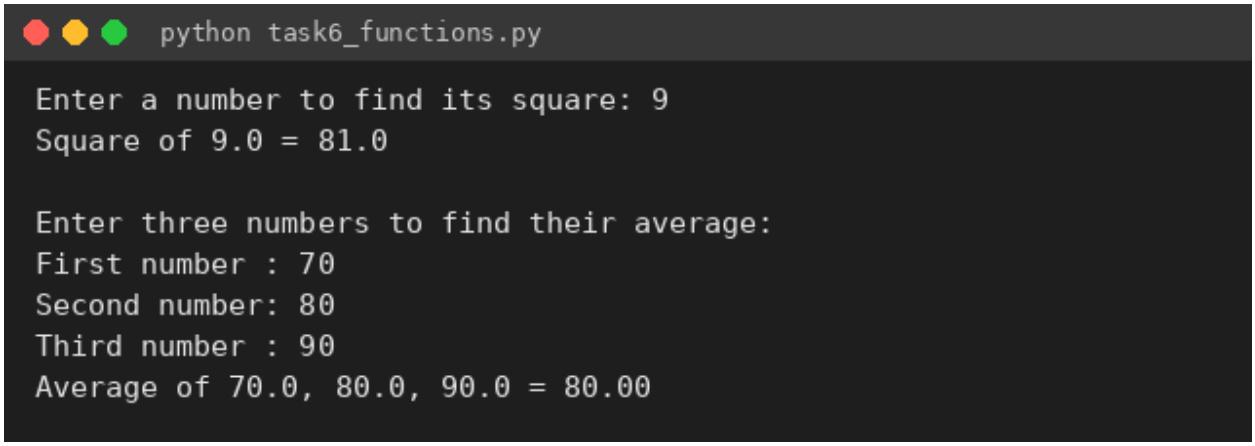
def square(num):
    """Return the square of a number."""
    return num * num


def average(a, b, c):
    """Return the average of three numbers."""
    return (a + b + c) / 3


n = float(input("Enter a number to find its square: "))
print(f"Square of {n} = {square(n)}")

print("\nEnter three numbers to find their average:")
x = float(input("First number : "))
y = float(input("Second number: "))
z = float(input("Third number : "))
print(f"Average of {x}, {y}, {z} = {average(x, y, z):.2f}")
```

Output Screenshot



Task 7: Strings & Collections (15 Marks)

Explanation

Demonstrates string methods (upper, lower, replace, find), list methods (append, remove, sort), tuple creation and indexing/slicing, a dictionary of student information, and set methods (add, remove).

Source Code (task7_collections.py)

```
# Task 7: Strings & Collections

# ---------- Strings ----------
print("===== STRING OPERATIONS =====")
text = "Python for Data Analytics"
print("Original :", text)
print("upper()  :", text.upper())
print("lower()  :", text.lower())
print("replace():", text.replace("Python", "SQL"))
print("find()   :", text.find("Data"), "(index of 'Data')")

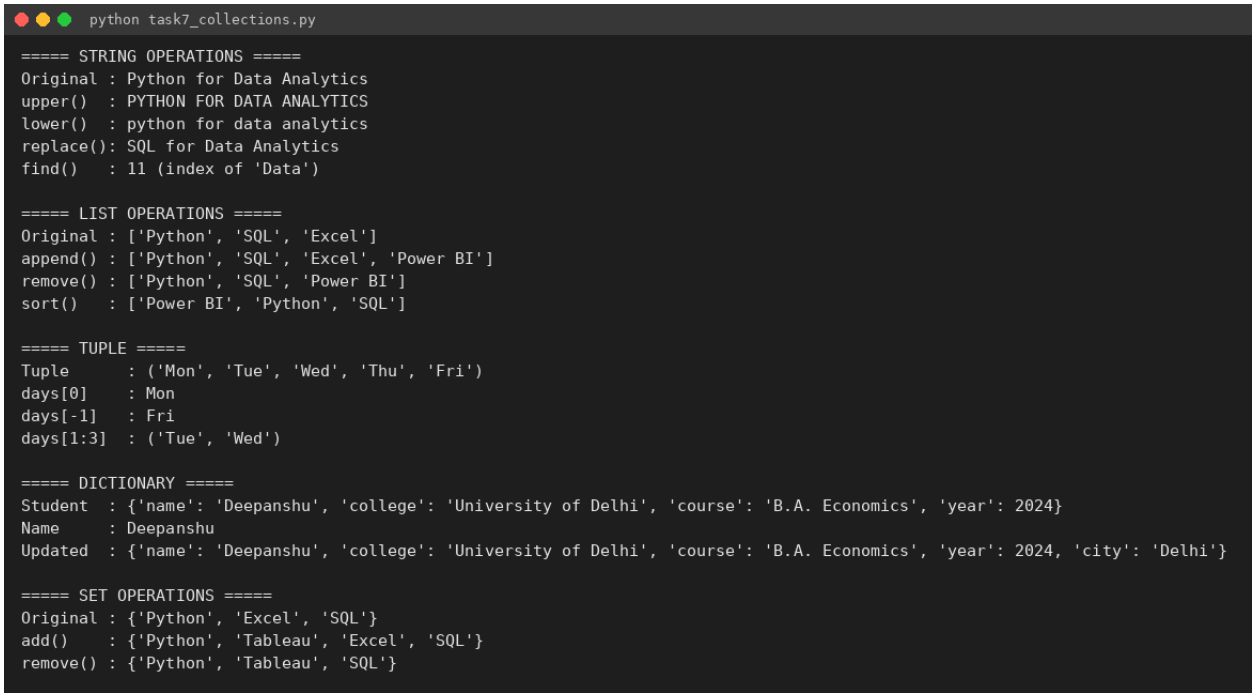
# ---------- Lists ----------
print("\n===== LIST OPERATIONS =====")
subjects = ["Python", "SQL", "Excel"]
print("Original :", subjects)
subjects.append("Power BI")
print("append() :", subjects)
subjects.remove("Excel")
print("remove() :", subjects)
subjects.sort()
print("sort()   :", subjects)

# ---------- Tuples ----------
print("\n===== TUPLE =====")
days = ("Mon", "Tue", "Wed", "Thu", "Fri")
print("Tuple      :", days)
print("days[0]    :", days[0])
print("days[-1]   :", days[-1])
print("days[1:3]  :", days[1:3])

# ---------- Dictionary ----------
print("\n===== DICTIONARY =====")
student = {
    "name": "Deepanshu",
    "college": "University of Delhi",
    "course": "B.A. Economics",
    "year": 2024,
}
print("Student  :", student)
print("Name     :", student["name"])
student["city"] = "Delhi"
print("Updated  :", student)

# ---------- Sets ----------
print("\n===== SET OPERATIONS =====")
skills = {"Python", "SQL", "Excel"}
print("Original :", skills)
skills.add("Tableau")
print("add()    :", skills)
skills.remove("Excel")
print("remove() :", skills)
```

Output Screenshot


python task7_collections.py
===== STRING OPERATIONS =====
Original : Python for Data Analytics
upper() : PYTHON FOR DATA ANALYTICS
lower() : python for data analytics
replace(): SQL for Data Analytics
find()   : 11 (index of 'Data')
===== LIST OPERATIONS =====
Original : ['Python', 'SQL', 'Excel']
append() : ['Python', 'SQL', 'Excel', 'Power BI']
remove() : ['Python', 'SQL', 'Power BI']
sort()   : ['Power BI', 'Python', 'SQL']
===== TUPLE =====
Tuple   : ('Mon', 'Tue', 'Wed', 'Thu', 'Fri')
days[0]   : Mon
days[-1]   : Fri
days[1:3] : ('Tue', 'Wed')
===== DICTIONARY =====
Student : {'name': 'Deepanshu', 'college': 'University of Delhi', 'course': 'B.A. Economics', 'year': 2024}
Name   : Deepanshu
Updated : {'name': 'Deepanshu', 'college': 'University of Delhi', 'course': 'B.A. Economics', 'year': 2024, 'city': 'Delhi'}
===== SET OPERATIONS =====
Original : {'Python', 'Excel', 'SQL'}
add()   : {'Python', 'Tableau', 'Excel', 'SQL'}
remove() : {'Python', 'Tableau', 'SQL'}

Task 8: Basic File Handling (10 Marks)

Explanation

Creates introduction.txt in write mode, writes a three-line introduction, then reopens it in read mode and prints the contents. The with statement closes the file automatically.

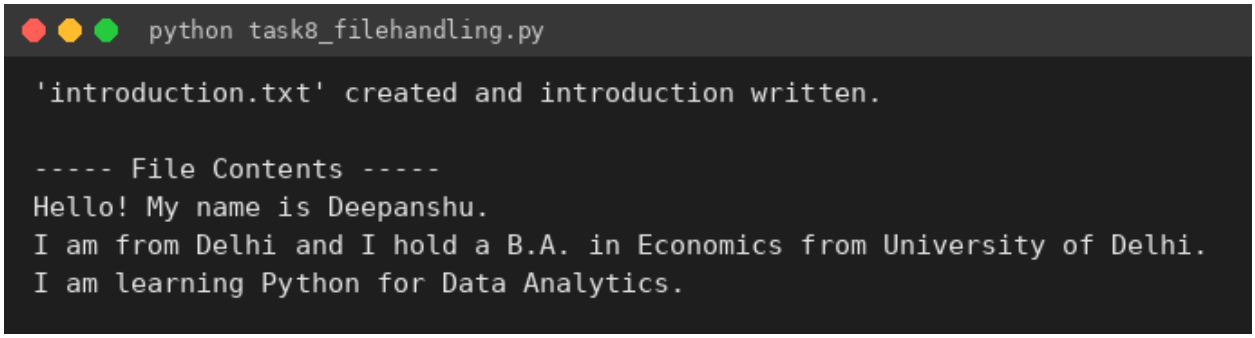
Source Code (task8_filehandling.py)

```
# Task 8: Basic File Handling
filename = "introduction.txt"

# 1 & 2. Create the file and write introduction
with open(filename, "w") as f:
    f.write("Hello! My name is Deepanshu.\n")
    f.write("I am from Delhi and I hold a B.A. in Economics from University of Delhi.\n")
    f.write("I am learning Python for Data Analytics.\n")
print(f"'{filename}' created and introduction written.\n")

# 3. Read and display the contents
print("----- File Contents -----")
with open(filename, "r") as f:
    print(f.read())
```

Output Screenshot



Task 9: Mini Project - Student Record Management System (15 Marks)

Explanation

A menu-driven program that stores students as dictionaries inside a list. It supports adding a student (duplicate roll numbers blocked), displaying all records in a table, searching by name (case-insensitive, partial match), and deleting a record. The sample run adds three students, searches, deletes one and shows the updated list.

Source Code (task9_student_records.py)

```
# Task 9: Mini Project - Student Record Management System
# Storage: list of dictionaries

students = []


def add_student():
    roll = input("Enter Roll No : ").strip()
    if any(s["roll"] == roll for s in students):
        print("A student with this Roll No already exists.")
        return
    name = input("Enter Name    : ").strip()
    course = input("Enter Course  : ").strip()
    marks = input("Enter Marks   : ").strip()
    students.append({"roll": roll, "name": name, "course": course, "marks": marks})
    print(f"Student '{name}' added successfully.")


def display_students():
    if not students:
        print("No records found.")
        return
    print(f"\n{'Roll':<8}{'Name':<18}{'Course':<18}{'Marks':<6}")
    print("-" * 50)
    for s in students:
        print(f"{s['roll']:<8}{s['name']:<18}{s['course']:<18}{s['marks']:<6}")


def search_student():
    key = input("Enter name to search: ").strip().lower()
    found = [s for s in students if key in s["name"].lower()]
    if found:
        print(f"\n{len(found)} record(s) found:")
        for s in found:
            print(f"  Roll: {s['roll']} | Name: {s['name']} | "
                  f"Course: {s['course']} | Marks: {s['marks']}")
    else:
        print("No student found with that name.")

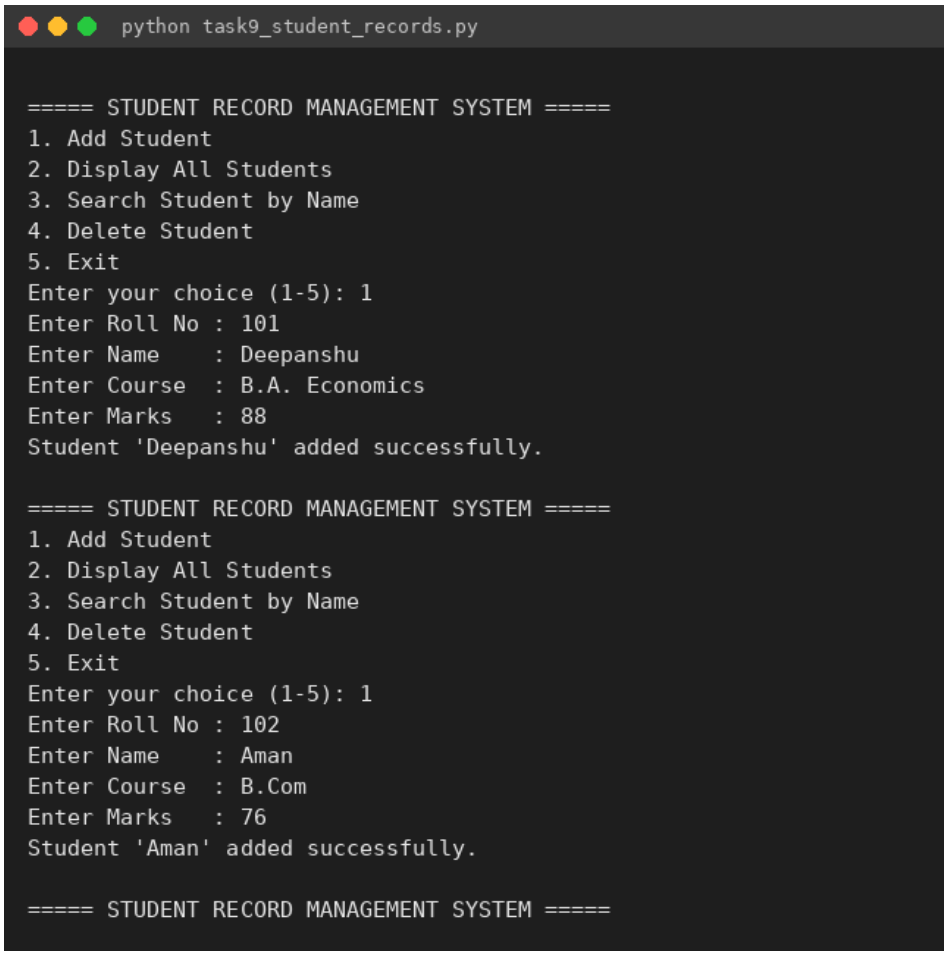

def delete_student():
    key = input("Enter name of student to delete: ").strip().lower()
    for s in students:
        if s["name"].lower() == key:
            students.remove(s)
            print(f"Record of '{s['name']}' deleted.")
            return
    print("Student not found.")


def main():
    while True:
        print("\n===== STUDENT RECORD MANAGEMENT SYSTEM =====")
        print("1. Add Student")
        print("2. Display All Students")
        print("3. Search Student by Name")
        print("4. Delete Student")
        print("5. Exit")
        choice = input("Enter your choice (1-5): ").strip()
```

```
        if choice == "1":
            add_student()
        elif choice == "2":
            display_students()
        elif choice == "3":
            search_student()
        elif choice == "4":
            delete_student()
        elif choice == "5":
            print("Thank you for using the system. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")


if __name__ == "__main__":
    main()
```

Output Screenshot



```
python task9_student_records.py

1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): 1
Enter Roll No : 103
Enter Name    : Riya
Enter Course  : B.Sc
Enter Marks   : 91
Student 'Riya' added successfully.

===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): 2

Roll    Name              Course            Marks
-------------------------------------------------
101     Deepanshu         B.A. Economics    88
102     Aman              B.Com             76
103     Riya              B.Sc              91

===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Add Student
```

```
python task9_student_records.py

2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): 3
Enter name to search: aman

1 record(s) found:
  Roll: 102 | Name: Aman | Course: B.Com | Marks: 76

===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): 4
Enter name of student to delete: riya
Record of 'Riya' deleted.

===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): 2
```

```
python task9_student_records.py

Roll    Name              Course            Marks
-------------------------------------------------
101     Deepanshu         B.A. Economics    88
102     Aman              B.Com             76

===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): 9
Invalid choice. Please try again.

===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): 5
Thank you for using the system. Goodbye!
```